

Splošna matura

Seminarska naloga pri računalništvu

KRMILJENJE, REGULACIJE, NADZOR PROCESOV

Nadgradnja daljinsko vodene kosilnice z računalniškim vidom in prostorskim umeščanjem

Mentor: Matjaž Majnik, univ. dipl. inž.

Avtor: Lin Čadež, G 4. A

Ljubljana, april 2026

Zahvala

Zahvaljujem se mentorju Matjažu Majniku, univ. dipl. inž., za pomoč in vodenje pri izdelavi seminarske naloge. Posebna zahvala gre mojemu očetu, ki mi je pomagal s finančno podporo pri nakupu GNSS modula in kamere PiCam ter za potrpežljivost in zaupanje med testiranjem mojih novih idej na njegovem mulčarju.

Povzetek

V seminarski nalogi sem predstavil nadgradnjo daljinsko vodene kosilnice z računalniškim vidom in prostorskim umeščanjem. Namen naloge je bil povečati varnost delovanja stroja na zahtevnem terenu, saj sistem ob zaznavi človeka ali živali samodejno zaustavi kosilnico.

Pri izdelavi sem uporabil računalnik Raspberry Pi 4B, kamero PiCam, knjižnico OpenCV, model MobileNet SSD ter GNSS-modul z DGPS-popravki prek protokola NTRIP. Razvil sem tudi spletni vmesnik za neposredno spremljanje videoprenosa in položaja kosilnice na zemljevidu.

Rezultati testiranja so pokazali, da sistem v ustreznih razmerah zanesljivo zazna človeka ali žival ter zaustavi kosilnico v manj kot 100 ms. Ugotovil sem tudi, da diferencialni GPS izboljša natančnost določanja položaja. Med omejitvami sistema ostajajo slabše delovanje v temi, manjša zanesljivost zaznave na večjih razdaljah in zakasnitev video prenosa.

Naloga dokazuje, da je mogoče z dostopnimi komponentami izdelati učinkovit varnostni sistem za nadgradnjo daljinsko vodenega delovnega stroja.

Ključne besede: daljinsko vodena kosilnica, računalniški vid, PWM, DGPS, NTRIP, Raspberry Pi

Kazalo

1	Uvod	6
2	Načrtovanje in izbira tehnologij	8
2.1	Daljinsko vodene kosilnice in varnostni izzivi	8
2.2	Elektronsko zaznavanje okolice pri delovnih strojih	8
2.3	Pregled izbranih tehnologij	9
2.3.1	Raspberry Pi 4B	9
2.3.2	OpenCV in MobileNet SSD	10
2.3.3	Knjižnica Picamera2	10
2.3.4	GNSS modul LC29H z DGPS podporo	10
2.3.5	NTRIP in RTK2go	11
2.3.6	Python in večnitnost	11
3	Predelava mulčarja	11
3.1	Analiza RC-krmiljenja in PWM-meritve	11
3.1.1	Varnostna preklopna logika	13
3.2	Načrt sistema in razvojne iteracije	14
3.2.1	Osnovna arhitektura sistema	14
3.2.2	Nastavitve za RC, kamero in GNSS	15
3.2.3	Glavna zanka programa	15
3.2.4	Strežnik in spletni vmesnik	16
3.3	Varnostni mehanizem samodejne zaustavitve	17
3.3.1	Sprotna zaustavitev ob zaznavi človeka	18
3.3.2	Zakasnitvena logika	18
3.3.3	Zaznava objektov z MobileNet SSD	18
3.3.4	Zmanjšanje zakasnitve video toka	19
3.4	GNSS, DGPS in NTRIP v praksi	20
3.4.1	Delovanje GPS in omejitve navadnih modulov	20
3.4.2	Diferencialni GPS in NTRIP	20
3.4.3	Vzpostavitev NTRIP-povezave in pošiljanje GGA	21
3.5	Spletni vmesnik za operaterja	22
3.6	Terensko testiranje in analiza rezultatov	22
3.6.1	Metodologija testiranja	22
3.6.2	Rezultati meritev	23
3.6.3	Analiza napak in omejitve sistema	23
4	Zaključek	26
	Viri in literatura	28

A Priloge	28
A.1 Video demonstracija sistema	28
A.2 Izvorna koda projekta	28

Slike

1	Izhodiščni mulčar, ki sem ga uporabil za nadgradnjo.	7
2	Vsi moduli sistema, povezani skupaj: GPS ščit, RC sprejemnik, kamera PiCam, Raspberry Pi in GPS antena.	9
3	Skica Raspberry Pi z označenimi pini in navedenimi funkcijami pinov, ki sem jih uporabljal pri povezovanju sistema.	10
4	Shema priključkov sprejemnika iz dokumentacije, ki sem jo uporabil pri načrtovanju GPIO-vezave.	12
5	Graf meritev PWM-signalov posameznih kanalov, izmerjenih z Arduino in Serial Plotterjem v Arduino IDE.	12
6	Daljiniec z označenimi gumbi in joysticki, ki sem jih testiral in jih uporabljam pri upravljanju kosilnice. Stikalo na vrhu služi za spremembo načina delovanja, joysticka pa za usmerjanje naprej, nazaj, levo in desno.	13
7	Prikaz principa delovanja diferencialnega GPS in popravkov NTRIP.	21
8	Spletni vmesnik z živim videoposnetkom kamere in GPS-lokacijo kosilnice na interaktivnem zemljevidu.	22
9	Testiranje sistema na terenu: mulčar od daleč s prenosnim računalnikom med preizkusi.	24
10	Notranjost mulčarja z dodanim računalnikom Raspberry Pi in nameščeno elektroniko.	25
11	Daljiniec, nameščen na mulčarju med terenskim testiranjem.	26

Seznam kratic in okrajšav

DGPS	Diferencialni GPS (<i>Differential Global Positioning System</i>)
GGA	Stavek NMEA s podatki o položaju GPS (<i>Global Positioning System Fix Data</i>)
GNSS	Globalni navigacijski satelitski sistem (<i>Global Navigation Satellite System</i>)
GPIO	Splošno-namenski vhodno-izhodni pini (<i>General Purpose Input/Output</i>)
GPS	Globalni sistem za določanje položaja (<i>Global Positioning System</i>)
HTML	Jezik za označevanje hiperteksta (<i>HyperText Markup Language</i>)
JSON	Notacija objektov JavaScript (<i>JavaScript Object Notation</i>)
MJPEG	Motion JPEG — standardni video format za pretakanje
NMEA	Standard za sporočila navigacijskih naprav (<i>National Marine Electronics Association</i>)
NTRIP	Omrežni prenos RTCM prek internetnega protokola (<i>Networked Transport of RTCM via Internet Protocol</i>)
PWM	Modulacija širine impulza (<i>Pulse Width Modulation</i>)
RC	Daljinsko krmiljenje (<i>Remote Control</i>)
RTCM	Standard za diferencialne GPS-popravke (<i>Radio Technical Commission for Maritime Services</i>)
RTK	Kinematika v realnem času (<i>Real-Time Kinematic</i>)
SSD	Enkratno zaznavanje (<i>Single Shot Detector</i>)
UART	Univerzalni asinhroni sprejemnik/oddajnik (<i>Universal Asynchronous Receiver/Transmitter</i>)

1 Uvod

Daljinsko vodene kosilnice se vse pogosteje uporabljajo za košnjo težje dostopnih terenov, kjer je klasična ročna košnja zahtevna ali nevarna. Opremljene so z ostrimi rezili, ki jih poganja bencinski motor, gibanje pa omogočajo električni motorji in goseničen pogon. Ukazi z daljinskega upravljalnika se do kosilnice prenašajo v obliki PWM-signalov (angl. *Pulse Width Modulation*, modulacija širine impulza) — to je način kodiranja ukazov z različno dolžino električnih impulzov. Kljub daljinskemu upravljanju ostaja odprto vprašanje, kako zagotoviti dodatno varnost in zmanjšati možnost nesreč, zlasti kadar se v bližini stroja znajdejo ljudje ali živali.

Za to seminarsko nalogo sem se odločil na podlagi osebne izkušnje z mulčarjem, ki ga upravljam na daljavo. Med delom sem razmišljal, kako bi bilo mogoče izboljšati varnost njegovega delovanja. Sprva sem preizkušal preproste ultrazvočne in temperaturne senzorje za zaznavanje ovir, vendar sem ugotovil, da niso primerni za ta namen: kosilnica namreč dela pretežno v zaraščenem in težko dostopnem terenu, na primer v praproti ali trnju, kjer je ves čas neka rasltina pred senzorjem in bi povzročila lažne zaznave. Takšno zaznavanje zato ni zanesljivo. Zato sem se odločil za uporabo kamere in računalniškega vida, ki omogočata razlikovanje med živimi bitji in neaktivnimi ovirami.

Namen seminarske naloge je predelava navadne daljinsko vodene kosilnice v varnejši sistem z zaznavanjem okolice in prostorsko umestitvijo. Cilj je razviti sistem, ki ob zaznavi človeka ali živali samodejno zaustavi kosilnico ter operaterju omogoča nadzor nad njenim delovanjem in položajem, pri čemer osnovno daljinsko upravljanje ostane nespremenjeno.



Slika 1: Izhodiščni mulčar, ki sem ga uporabil za nadgradnjo.

Cilji seminarske naloge so:

- analizirati delovanje daljinskega upravljanja kosilnice in PWM-signalov,
- izbrati primeren sistem za zaznavanje ljudi in živali z uporabo kamere,
- razviti varnostni mehanizem za samodejno zaustavitev kosilnice,
- preizkusiti prostorsko umestitev kosilnice z uporabo diferencialnega GPS,
- izdelati osnovni spletni vmesnik za spremljanje delovanja sistema.

Hipoteza seminarske naloge je, da je z uporabo računalniškega vida mogoče zanesljivo zaznati ljudi in živali v okolici kosilnice ter s tem preprečiti morebitne poškodbe. Pričakujem, da bo odzivni čas od zaznave do zaustavitve krajši od 700 ms, kar sem postavil kot praktično mejo za varno delovanje.

Raziskovalne metode so temeljile na eksperimentalnem delu, meritvah signalov, praktičnih preizkusih delovanja sistema in analizi rezultatov testiranj na terenu.

2 Načrtovanje in izbira tehnologij

2.1 Daljinsko vodene kosilnice in varnostni izzivi

Daljinsko vodene kosilnice so posebna vrsta delovnih strojev, namenjenih košnji trave in vzdrževanju površin, kjer bi bila klasična košnja z ročno ali traktorsko kosilnico neugodna oziroma celo nevarna, na primer na strmih pobočjih, v trnju ali na drugih težje dostopnih terenih. Upravljanje poteka z brezžičnim daljinskim upravljalnikom, kar omogoča, da se operater nahaja na varni razdalji od stroja.

Delovanje daljinsko vodenih kosilnic temelji na kombinaciji mehanskega in električnega pogona. Rezalni sistem običajno poganja bencinski motor, medtem ko za premikanje in krmiljenje pogosto skrbijo elektromotorji, ki delujejo na gosenični ali kolesni pogon. Ukazi z daljinskega upravljalnika se do kosilnice prenašajo v obliki PWM-signalov, ki določajo smer, hitrost gibanja ter druge funkcije stroja.

Kljub daljinskemu upravljanju predstavljajo takšne kosilnice resne varnostne izzive. Največjo nevarnost predstavljajo hitro vrteča se rezila, ki lahko ob stiku povzročijo hude telesne poškodbe.

2.2 Elektronsko zaznavanje okolice pri delovnih strojih

Pri delovanju avtonomnih ali daljinsko vodenih strojev je zanesljivo zaznavanje okolice ključno za varno navigacijo in izogibanje oviram. Razlikujemo več vrst senzorjev, ki se razlikujejo po dosegu, natančnosti in zmožnosti razlikovanja med vrstami objektov.

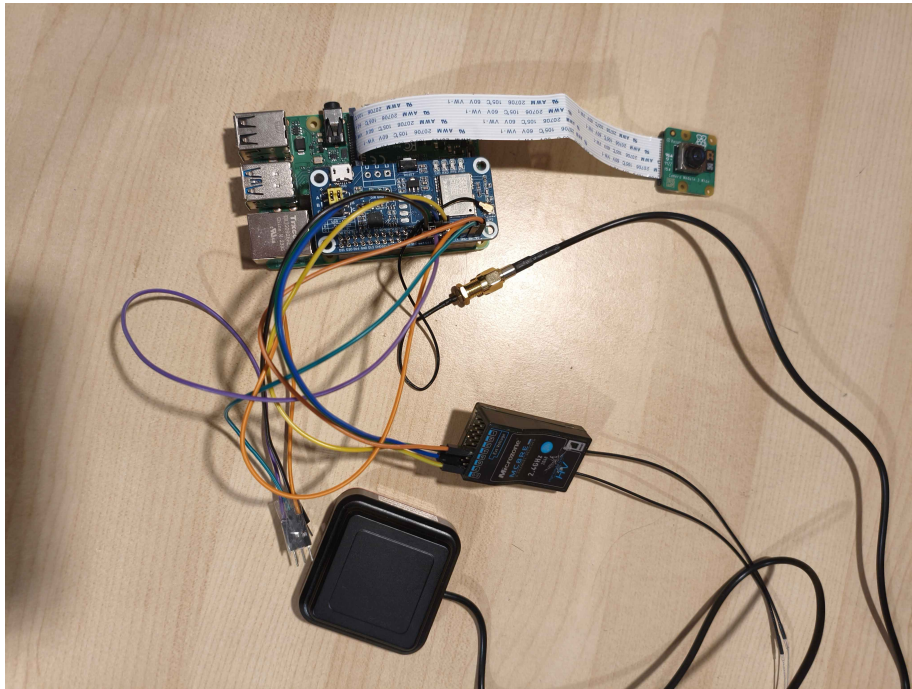
Ultrazvočni senzorji oddajajo zvočne valove nad človeškim slušnim območjem in merijo čas, ki ga signal potrebuje, da se odbije od ovire in vrne do sprejemnika. So cenovno ugodni in enostavni za vgradnjo, vendar imajo za ta namen ključno pomanjkljivost: ne ločijo med vrsto objekta (rastlina, človek, žival) in v gosto zaraščenem terenu preprosto ne delujejo zaradi vegetacije pred senzorjem. Posledično bi kosilnica v takšnih razmerah nenehno delala varnostne prekinitve, četudi ni v bližini nobenega živega bitja, ki bi ga morebiti ogrožala.

Infrardeči senzorji zaznavajo spremembe v infrardečem svetlobnem spektru in so primerni za zaznavanje premikanja ter prisotnosti živih bitij. Njihova ključna pomanjkljivost je občutljivost na temperaturne razmere: pri poletni košnji so zunanje temperature včasih primerljive s temperaturo človeškega telesa, kar zmanjša zanesljivost razlikovanja med živimi bitji in segreto okolico.

Kamerični sistemi zajemajo vizualne podatke, ki jih procesira programska oprema. Kamere

omogočajo bistveno bogatejše informacije o okolju kot enostavni senzorji, saj omogočajo detekcijo, sledenje in razlikovanje med različnimi vrstami ovir. Takšne zmogljivosti so ključne pri nalogah, kjer je treba razlikovati med živimi bitji in neživimi predmeti.

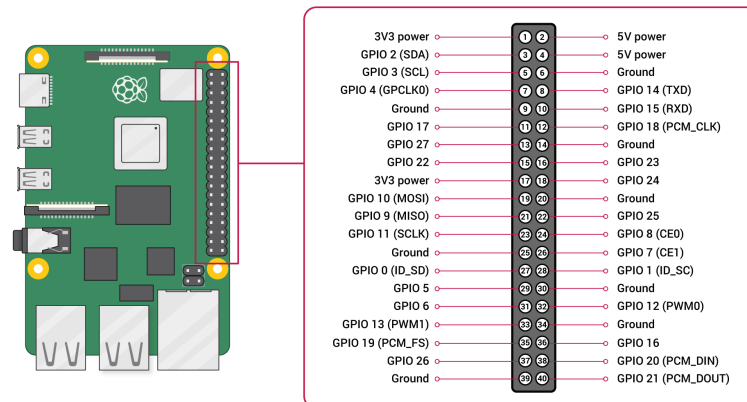
2.3 Pregled izbranih tehnologij



Slika 2: Vsi moduli sistema, povezani skupaj: GPS ščit, RC sprejemnik, kamera PiCam, Raspberry Pi in GPS antena.

2.3.1 Raspberry Pi 4B

Raspberry Pi 4B je majehn računalnik z vgrajenimi GPIO-priključki, CSI-vmesnikom za kamero, modulom Wi-Fi in operacijskim sistemom Linux (Raspberry Pi OS). Izbral sem ga, ker je dovolj majhen, da sem ga lahko namestil neposredno v kosilnico. Za njegovo delovanje zadostuje napetost 5 V, zato sem ga lahko priključil na matično ploščo kosilnice, kjer sta bili na voljo napetosti 5 V in 12 V. Pomembna prednost je bila tudi dobra podpora za Python, OpenCV in delo s kamero, kar mi je bistveno olajšalo razvoj sistema.



Slika 3: Skica Raspberry Pi z označenimi pini in navedenimi funkcijami pinov, ki sem jih uporabljal pri povezovanju sistema.

2.3.2 OpenCV in MobileNet SSD

OpenCV je odprtokodna knjižnica za računalniški vid, ki med drugim vsebuje modul `dnn` za poganjanje nevronske mreže (OpenCV Team, 2024). Za prepoznavanje objektov sem izbral med modelom YOLO, ki je natančnejši, vendar računsko precej potraten, in modelom MobileNet SSD (Howard in sod., 2017), ki je bil posebej zasnovan za naprave z omejeno procesorsko močjo. Na Raspberry Pi je bil MobileNet SSD bistveno hitrejši in je dosegal zadovoljivo hitrost prepoznavanja. Nekatere težave z natančnostjo so se pojavljale pri testih v temi, vendar se kosilnica v praksi uporablja podnevi, zato to ni kritično.

2.3.3 Knjižnica Picamera2

Knjižnica Picamera2 je uradna Python knjižnica za upravljanje kamere na Raspberry Pi (Raspberry Pi Ltd., 2023). V primerjavi s starejšo knjižnico PiCamera bolje podpira novejša kamerne module in se brezhibno integrira z OpenCV, kar omogoča neposreden prenos okvirjev v obliki NumPy matrik brez dodatnih konverzij. To je pomenilo manj dela pri pisanju kode za zajem slike.

2.3.4 GNSS modul LC29H z DGPS podporo

Za natančno pozicioniranje sem izbral modul LC29H podjetja Quectel (*LC29H (EA&DA) GNSS Module Series – Hardware Design Guide*, 2023), ki podpira sprejem signalov iz več satelitskih konstelacij (GPS, GLONASS, Galileo, BeiDou) in sprejem RTCM-diferencialnih popravkov prek UART-vmesnika. Navadni ceneni GPS-moduli dosegajo natančnost približno

± 5 m, moji testi z modulom NEO-7M pa so pokazali, da se natančnost v neugodnih razmerah poslabša tudi na 30 m ali več. Modul LC29H z DGPS-popravki dosega natančnost pod 2 m, kar je ključno za smiselno lociranje kosilnice pri delu na daljavo.

2.3.5 NTRIP in RTK2go

NTRIP (*Networked Transport of RTCM via Internet Protocol*) je standardiziran protokol za prenos diferencialnih GPS-popravkov prek interneta v realnem času (Federal Agency for Cartography and Geodesy (BKG), 2024). Za popravke sem uporabil javno dostopno storitev RTK2go in referenčno postajo FRELIH v Črnem Vrhu nad Idrijo (SNIP / RTK2go Community, 2024). Testiranja sem izvajal v Cerknem, ki je od postaje oddaljeno približno 33 km. Sicer bi bilo bolje, če bi bila referenčna postaja bližje, vendar sem bil med testiranjem zadovoljen z doseženo natančnostjo, ki je znašala približno $\pm 1,5$ m.

2.3.6 Python in večnitnost

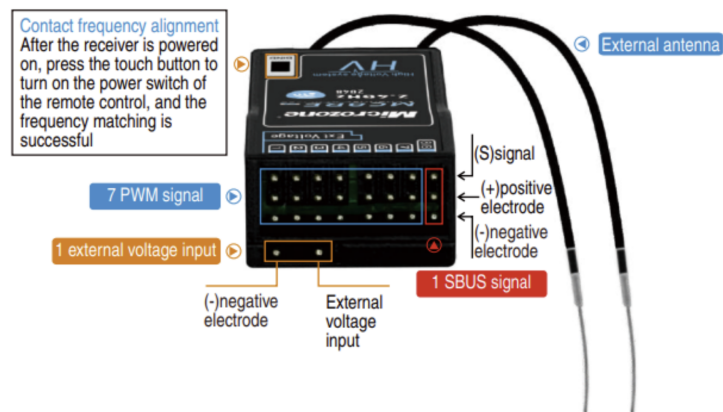
Celoten sistem sem implementiral v Pythonu 3, ki je standardni jezik za razvoj na Raspberry Pi. Posamezne naloge, kot so branje kamere, video strežnik, branje GPS, pošiljanje NTRIP-podatkov in krmilna zanka, tečejo v ločenih nitih z modulom `threading`. Tak pristop omogoča, da sistem več opravil izvaja hkrati, kar je bistveno za dovolj hiter odziv v primeru ovire.

3 Predelava mulčarja

3.1 Analiza RC-krmiljenja in PWM-meritve

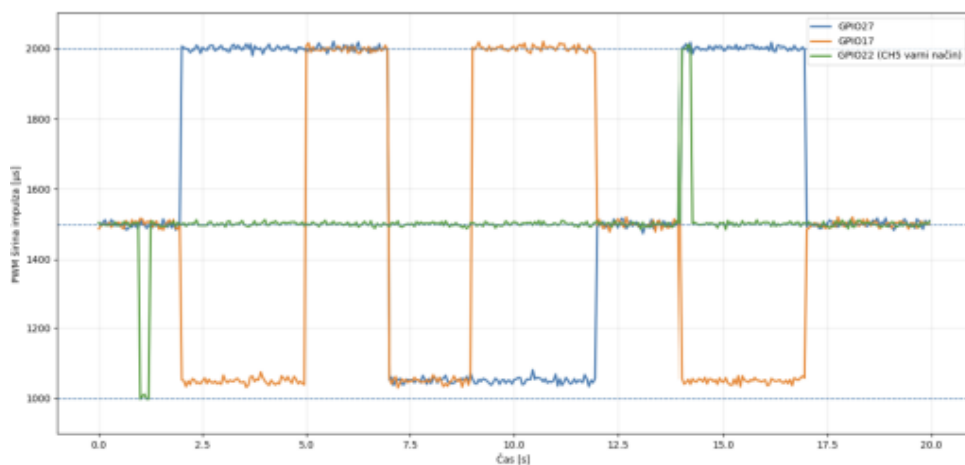
Ko sem začel z nadgradnjo, sem najprej preučil dokumentacijo sprejemnika, ki je že vgrajen v kosilnico.

FUNCTIONAL PARSING



Slika 4: Shema priključkov sprejemnika iz dokumentacije, ki sem jo uporabil pri načrtovanju GPIO-vezave.

Nato sem opravil meritve z mikrokrmilnikom Arduino Uno, ki mi je služil kot neke vrste oscilator. Napisal sem program, ki je prek analognih vhodov meril, s kakšno frekvenco sprejemnik oddaja PWM-signal, prejet od daljince. Podatke sem prikazal v programu Arduino Serial Plotter znotraj razvojnega okolja Arduino IDE.



Slika 5: Graf meritev PWM-signalov posameznih kanalov, izmerjenih z Arduino in Serial Plotterjem v Arduino IDE.

Na podlagi meritev sem ugotovil, da se posamezna smer premikanja kosilnice prepozna po značilnih pulznih širinah na dveh kanalih. Vizualizacija poteka signalov v realnem času mi je omogočila natančno določanje mejnih vrednosti za premikanje naprej, nazaj, levo in desno ter določitev nevtralnega položaja.

V spodnji tabeli so zapisane vrednosti, ki sem jih eksperimentalno določil s premikanjem "joystickov" na daljincu.



Slika 6: Daljinec z označenimi gumbi in joysticki, ki sem jih testiral in jih uporabljam pri upravljanju kosilnice. Stikalo na vrhu služi za spremembo načina delovanja, joysticka pa za usmerjanje naprej, nazaj, levo in desno.

Tabela 1: Izmerjene PWM-vrednosti za osnovne položaje krmiljenja

Položaj	CH2 / GPIO 27	CH1 / GPIO 17
Naprej	2000 μs	1050 μs
Mirovanje	1500 μs	1500 μs
Nazaj	1050 μs	2000 μs
Levo	2000 μs	2000 μs
Desno	1050 μs	1050 μs

3.1.1 Varnostna preklopna logika

V glavni zanki preverjam, ali je vrednost kanala CH5 — ki je vezan na stikalo na daljincu — večja od konstante `AUTOPILOT_ON`. Osnovno stanje digitalnega vhoda sicer lahko preberemo z metodo *digital read* (na primer `pi.read(CH5_IN)` ali `GPIO.input(CH5_IN)`), vendar tak pristop zazna le logični stanji HIGH ali LOW.

Ker RC-sprejemnik ne oddaja klasičnega digitalnega signala, temveč PWM-signal, sem za določanje položaja stikala uporabil merjenje širine impulza. Dolžino posameznega impulza izračunam z uporabo funkcije `pigpio.tickDiff(_last_tick[gpio], tick)`, ki določi časovno razliko med naraščajočim in padajočim robom signala.

V programu sem nastavljal mejo za vklop na $1900\ \mu s$, mejo za izklop na $1100\ \mu s$, nevtralno vrednost pa je približno $1500\ \mu s$.

```
if ch5 > AUTOPILOT_ON and not autopilot_mode:
    autopilot_mode = True
elif ch5 < AUTOPILOT_OFF and autopilot_mode:
    autopilot_mode = False
```

Izsek kode 1: Varnostna preklompna logika glede na kanal CH5.

3.2 Načrt sistema in razvojne iteracije

3.2.1 Osnovna arhitektura sistema

Najprej sem v glavno datoteko uvozil vse potrebne knjižnice, ki olajšajo programiranje posameznih delov sistema.

```
import base64
import json
import socket
import threading
import time

from http.server import HTTPServer, BaseHTTPRequestHandler

import serial
import cv2
from picamera2 import Picamera2
import pigpio
```

Izsek kode 2: Osnovna struktura in knjižnice v final.py.

Posamezne knjižnice v programu opravljajo različne naloge. Knjižnica `base64` se uporablja za kodiranje podatkov pri vzpostavitvi NTRIP-povezave, `json` za oblikovanje podatkov v JSON-format za spletni vmesnik, `socket` pa za omrežno komunikacijo s strežnikom NTRIP.

Knjižnica `time` služi za merjenje časa in izvajanje časovnih zamikov v programu.

Razred `HTTPServer` skupaj z `BaseHTTPRequestHandler` omogoča izdelavo preprostega spletnega strežnika za prikaz podatkov in uporabniškega vmesnika.

Knjižnica `serial` se uporablja za komunikacijo z GNSS-modulom prek serijskega vmesnika. Knjižnica `cv2` (OpenCV) omogoča obdelavo slike in zaznavanje objektov, `Picamera2` pa skrbi za zajem slike s kamere Raspberry Pi.

Knjižnica `pigpio` omogoča natančno branje in generiranje PWM-signalov na GPIO-pinih, kar je ključno za komunikacijo z RC-sprejemnikom in krmiljenje kosilnice.

Knjižnica `threading` omogoča večnitno izvajanje programa, kar pomeni, da lahko sistem sočasno izvaja več nalog, kot so zajem slike s kamere, obdelava slike, komunikacija z GNSS-modulom in delovanje spletnega strežnika.

3.2.2 Nastavitve za RC, kamero in GNSS

Tu sem definiriral osnovne konstante in preklopne meje na enem mestu, ker sem jih med testiranjem večkrat spreminjal. V `TARGETS` sem vpisal imena objektov, ki jih nevronska mreža zazna na sliki. Z dodajanjem drugih imen objektov bi lahko dosegli, da bi se kosilnica ustavila tudi ob zaznavi drugačnih predmetov, vendar to za cilj moje naloge ni bilo potrebno.

```
CH1_IN = 17;  CH2_IN = 27;  CH5_IN = 22
CH1_OUT = 23; CH2_OUT = 24

MIN_PULSE = 900;  MAX_PULSE = 2100;  NEUTRAL_PULSE = 1500
AUTOPILOT_ON = 1900;  AUTOPILOT_OFF = 1100
CONF_THRESH = 0.5
TARGETS = {"person", "cat", "dog"}
```

Izsek kode 3: Ključne nastavitve za signalne pragove in zaznavo.

3.2.3 Glavna zanka programa

Glavna zanka programa povezuje vse podsisteme v enoten sistem. V funkciji `main()` se najprej odpre serijska povezava z GNSS-modulom, nato pa se za posamezne naloge zaženejo ločene niti. Ena nit skrbi za branje GNSS-podatkov, druga za povezavo z NTRIP-strežnikom, tretja za zajem slike s kamere, četrta za video tok, dodatna nit pa za spletni strežnik uporabniškega vmesnika. Na koncu se zažene `control_loop()`, ki neprekinjeno izvaja glavno krmilno logiko.

```
def main():
    ser = serial.Serial(SERIAL_PORT, SERIAL_BAUD, timeout=1)
    print("[INFO] GNSS serial open:", SERIAL_PORT, SERIAL_BAUD)

    threading.Thread(target=gnss_reader_thread,
                     args=(ser,), daemon=True).start()
    threading.Thread(target=ntrip_manager_thread,
                     args=(ser,), daemon=True).start()
```

```

threading.Thread(target=camera_thread, daemon=True).start()
threading.Thread(target=mjpeg_stream_thread, daemon=True).start()

threading.Thread(
    target=lambda: HTTPServer(
        ("0.0.0.0", HTTP_PORT), DashboardHandler
    ).serve_forever(),
    daemon=True
).start()

control_loop()

```

Izsek kode 4: Glavna funkcija programa in zagon vseh niti.

Takšna zasnova omogoča, da sistem več funkcij izvaja hkrati in da posamezen podsistem ne blokira delovanja ostalih.

3.2.4 Strežnik in spletni vmesnik

Pomemben del sistema je bil tudi spletni strežnik, ki operaterju omogoča dostop do slike s kamere in do trenutnega položaja kosilnice. Strežnik je zasnovan tako, da ob obisku naslova / vrne HTML-stran z vmesnikom, ob zahtevi na naslov /pos pa vrne trenutne GPS-podatke v obliki JSON. Na ta način lahko brskalnik sproti osvežuje položaj kosilnice in ga prikazuje na zemljevidu.

```

class DashboardHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/" or self.path == "/index.html":
            self.send_response(200)
            self.send_header("Content-Type",
                             "text/html; charset=utf-8")
            self.end_headers()
            self.wfile.write(INDEX_HTML)
            return

        if self.path == "/pos":
            with state_lock:
                data = json.dumps(latest).encode("utf-8")
            self.send_response(200)
            self.send_header("Content-Type",
                             "application/json; charset=utf-8")

```

```
self.end_headers()
self.wfile.write(data)
return
```

Izsek kode 5: Osnovni del strežnika za pošiljanje HTML-strani in GPS-podatkov.

Na strani odjemalca sem uporabil JavaScript, ki vsakih 500 milisekund pošilja zahteve na `/pos` in posodablja položaj na zemljevidu. Hkrati se nalaga video tok z naslova `/stream`, zato lahko operater na enem mestu spremlja sliko in lokacijo.

```
async function pollPos() {
  try {
    const r = await fetch('/pos', { cache: 'no-store' });
    if (!r.ok) throw new Error('HTTP ' + r.status);
    const d = await r.json();

    if (d.lat == null || d.lon == null) {
      posPill.textContent = '/pos: no data';
      return;
    }

    marker.setLatLng([d.lat, d.lon]);
  } catch (e) {
    posPill.textContent = '/pos: ERROR';
  }
}

setInterval(pollPos, 500);
```

Izsek kode 6: Periodično pridobivanje položaja in posodabljanje zemljevida.

3.3 Varnostni mehanizem samodejne zaustavitve

Pri varnostnem podsistemu sem najprej naredil osnovno različico, ki je kosilnico ustavila takoj, ko je kamera zaznala človeka, psa ali mačko. Opazil sem, da je sistem včasih zaznal oviro le v enem okvirju, zato se kosilnica ni vedno ustavila dovolj zanesljivo. Dodal sem časovni zamik: kosilnica po zaznavi ovire obstane še eno sekundo in šele nato preveri, ali je objekt še vedno prisoten. Če ni, jo lahko naprej upravljamo z ukazi z RC daljinca.

3.3.1 Sprotna zaustavitev ob zaznavi človeka

Ko je varnostni režim vključen, morata motorja takoj dobiti nevtralni pulz. Ker program deluje v več nitih hkrati, sem zagotovil, da se kosilnica ustavi ne glede na trenutno obremenitev drugih delov sistema. V glavni krmilni zanki ob zaznavi cilja ali ob aktivnem varnostnem zadrževanju obema izhodoma takoj nastavim nevtralno vrednost, kar zagotavlja hiter in zanesljiv odziv.

```
if autopilot_mode and (target_detected or safety_latched):
    pi.set_servo_pulsewidth(CH1_OUT, NEUTRAL_PULSE)
    pi.set_servo_pulsewidth(CH2_OUT, NEUTRAL_PULSE)
else:
    pi.set_servo_pulsewidth(CH1_OUT, ch1)
    pi.set_servo_pulsewidth(CH2_OUT, ch2)
```

Izsek kode 7: Samodejna zaustavitev ob zaznanih preprekah.

3.3.2 Zakasnitvena logika

Program v spremenljivko zapiše čas, ko je bila ovira nazadnje zaznana, nato pa preverja, ali je od tega trenutka pretekla vsaj ena sekunda. Če ovire v tem času ni več, se varnostni mehanizem sprosti in program nadaljuje normalno delovanje. S tem sem preprečil, da bi se sistem ob kratkotrajni prekinitvi zaznave prehitro ponovno zagnal.

```
if target_detected:
    safety_latched = True
    target_clear_since = None
else:
    if safety_latched:
        if target_clear_since is None:
            target_clear_since = now
        elif (now - target_clear_since) >= CLEAR_DELAY_S:
            safety_latched = False
            target_clear_since = None
```

Izsek kode 8: 1-sekundni zamik pri ponovni sprostitvi varnostne zaustavitve.

3.3.3 Zaznava objektov z MobileNet SSD

Odločil sem se za MobileNet SSD, ker sem ugotovil, da je za moj primer uporabe pomembnejša hitrost kot natančnost. YOLO sem preizkusil, vendar je bila prepoznava predolga

(približno dve sekundi), kar bi v praksi pomenilo prepozen odziv. Sliko pri prepoznavi zmanjšam na 160×160 pikslov, saj je obdelava tako hitrejša. Glede na teste se natančnost zaradi tega ni bistveno zmanjšala.

```
small = cv2.resize(current_frame, (160, 160))
blob = cv2.dnn.blobFromImage(small, 0.007843,
                              (160, 160), 127.5)

net.setInput(blob)
detections = net.forward()

detected_now = any(
    float(detections[0, 0, i, 2]) >= CONF_THRESH and
    CLASSES[int(detections[0, 0, i, 1])] in TARGETS
    for i in range(detections.shape[2])
)
```

Izsek kode 9: Zaznavanje tarč z MobileNet SSD.

3.3.4 Zmanjšanje zakasnitve video toka

Video tok sem namensko stisnil in zmanjšal kakovost slike, ker sem ugotovil, da je za vodenje kosilnice na daljavo pomembnejša odzivnost kot ostrina vsakega okvirja. Sprva se je slika posodabljala približno vsakih 1,5 s; po zmanjšanju ločljivosti in omogočenju kompresije sem zakasnitev zmanjšal na približno 0,5 s, kar je bilo dovolj hitro za varno vodenje na daljavo.

```
ok, jpeg = cv2.imencode('.jpg', current_frame,
                        [cv2.IMWRITE_JPEG_QUALITY, 45])

if ok:
    self.wfile.write(b'--frame\r\n')
    self.wfile.write(b'Content-Type: image/jpeg\r\n\r\n')
    self.wfile.write(jpeg.tobytes())
    self.wfile.write(b'\r\n')
```

Izsek kode 10: MJPEG-strežnik z zmerno kakovostjo JPEG.

3.4 GNSS, DGPS in NTRIP v praksi

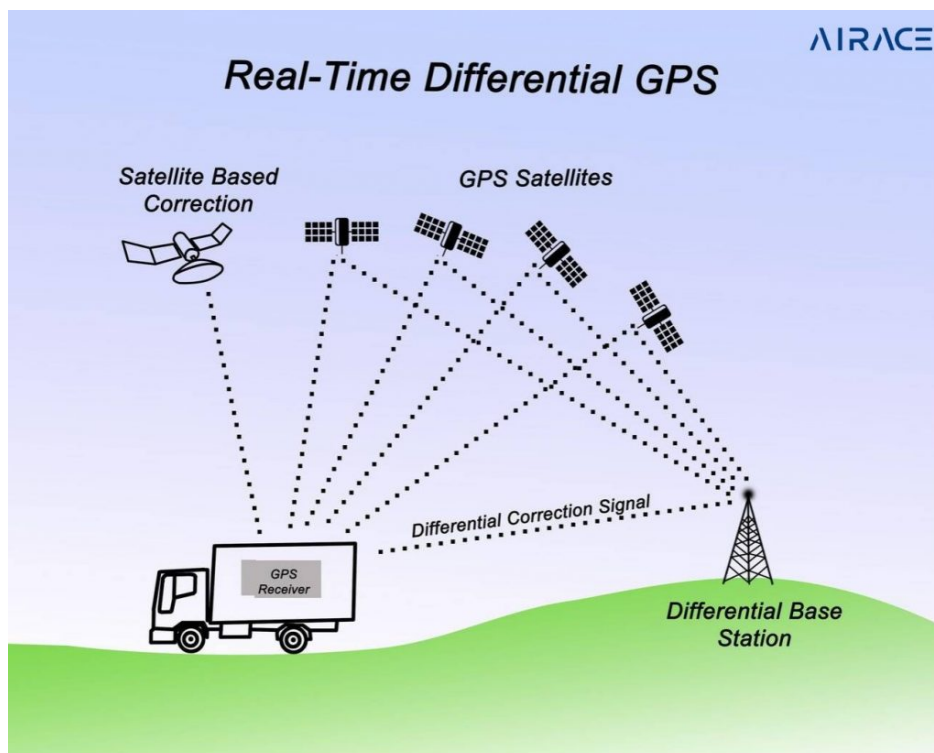
3.4.1 Delovanje GPS in omejitve navadnih modulov

GPS deluje na podlagi izmere časa, ki ga signal potrebuje, da potuje od satelita do sprejemnika. Ker signal potuje s svetlobno hitrostjo, sprejemnik iz razlik v časih sprejema od vsaj štirih satelitov izračuna svoj prostorski položaj. Natančnost je odvisna od lokacije satelitov, atmosferskih pojavov, odbojev signalov in kakovosti sprejemnika. Evropska vesoljska agencija v svoji dokumentaciji poudarja, da je za višjo natančnost potrebna dodatna korekcijska plast (European Space Agency, 2024).

Navadni ceneni GPS-moduli dosegajo natančnost okrog ± 5 m, ki pa se v neugodnih razmerah glede na moje teste z modulom NEO-7M poslabša na 30 m ali več. Za kosilnico, ki jo operater vodi na daljavo in je morda ne vidi neposredno, je takšna nenatančnost položaja potencialno nevarna, saj operater ne more zanesljivo oceniti, kje je stroj.

3.4.2 Diferencialni GPS in NTRIP

Diferencialni GPS (DGPS) deluje tako, da referenčna postaja na znani lokaciji neprestano meri napako GPS-a in te popravke v obliki RTCM-sporočil pošilja sprejemniku v terenu (Federal Agency for Cartography and Geodesy (BKG), 2024). Sprejemnik nato odšteje izmerjeno napako in dobi bistveno bolj natančne GPS položaj.



Slika 7: Prikaz principa delovanja diferencialnega GPS in popravkov NTRIP.

Za popravke sem uporabil javno dostopno storitev RTK2go in referenčno postajo FRELIH v Črnem Vrhu nad Idrijo (SNIP / RTK2go Community, 2024). Testiranja sem izvajal v Cerknem, ki je od postaje oddaljeno okrog 33 km.

3.4.3 Vzpostavitev NTRIP-povezave in pošiljanje GGA

Sprejemnik prek serijskega vmesnika pošilja NMEA-stavke, med katerimi je ključen GGA (*Global Positioning System Fix Data*). Tega pošiljamo nazaj na NTRIP-caster, ki v nazaj vrne RTCM-korekcijske podatke. Za takšno delovanje je potrebna stalna internetna povezava; med testiranjem sem sistem priključil na mobilno dostopno točko prek pametnega telefona.

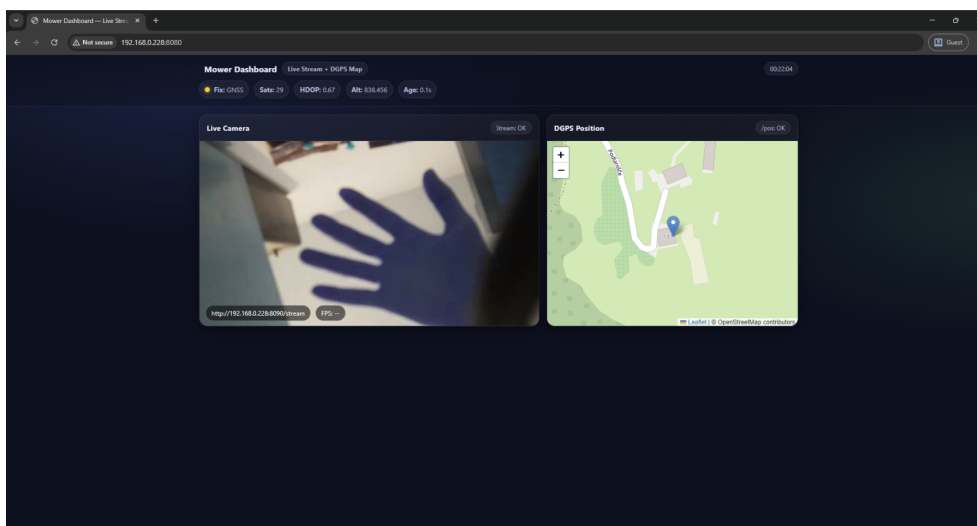
```
sock = socket.create_connection((NTRIP_HOST, NTRIP_PORT),
                                timeout=10)

auth = base64.b64encode(
    f"{USERNAME}:{PASSWORD}".encode()).decode()
req = (
    f"GET /{MOUNTPOINT} HTTP/1.0\r\n"
    f"User-Agent: Python-NTRIP\r\n"
    f"Authorization: Basic {auth}\r\n\r\n"
)
sock.sendall(req.encode("ascii", errors="ignore"))
```

Izsek kode 11: Vzpostavitev povezave na NTRIP-caster in pošiljanje GGA-stavka.

3.5 Spletni vmesnik za operaterja

Spletni vmesnik sem izdelal zato, da sem sistem lahko spremljal brez neposrednega priklopa s kablom na Raspberry Pi. Kosilnica v živo pretaka video s kamere in pošilja svoj položaj, ki se prikaže na odprtokodnem zemljevidu Leaflet. Vmesnik sem razvil v HTML-ju in JavaScriptu brez dodatnih strežniških ogrodij, ker sem želel, da sistem ostane čim enostavnejši za vzdrževanje.



Slika 8: Spletni vmesnik z živim videoposnetkom kamere in GPS-lokacijo kosilnice na interaktivnem zemljevidu.

3.6 Terensko testiranje in analiza rezultatov

3.6.1 Metodologija testiranja

Preden sem združil vse podsisteme v končno skripto `final.py`, sem moral vsak del preizkusiti ločeno. Zato sem uporabil ločene testne skripte za kamero, prikaz PWM-signalov ter preizkus NTRIP-povezave. Tak pristop mi je omogočil, da sem napake odkril pravočasno, preden so postale del večje celote.

Med terenskim testiranjem sem sistem preizkušal na asfaltu pred hišo. Od kosilnice sem stal na razdalji od 1 do 3 m in jo upravljal prek daljinca, medtem ko sem se postavil pred kosilnico in upal, da me ne povozi. Med testiranjem so bila rezila izklopljena.

3.6.2 Rezultati meritev

Sistem je zaustavil kosilnico v manj kot 100 ms od zaznave osebe pri razdalji do 4 m in zadostnih svetlobnih razmerah. GPS-natančnost je bila z DGPS-popravki stabilna pod 1 m, v primerjavi s ± 5 m ali več brez popravkov. Spodnja tabela povzema ključne izmerjene vrednosti.

Tabela 2: Izmerjene vrednosti ključnih parametrov sistema

Parameter	Izmerjena vrednost	Zastavljeni cilj
Odzivni čas (zaznava → zaustavitev)	< 100 ms	< 700 ms
Zakasnitev video toka	$\approx 1,0$ s	< 0,5 s
GPS-natančnost (brez DGPS)	$\pm 5,0$ m	—
GPS-natančnost (z DGPS modulom)	$\pm 1,5$ m	< 1,0 m
Največja zanesljiva razdalja zaznave	≈ 4 m	—

3.6.3 Analiza napak in omejitve sistema

Med testiranjem sem naletel na več nevšečnosti.

Zakasnitev video toka. Na začetku je bila zakasnitev približno 1,5 s zaradi previsoke ločljivosti slike. Po optimizaciji sem jo zmanjšal na približno 1,0 s, kar je za uporabo še sprejemljivo, čeprav ne dosega zastavljenega cilja.

Zaznava v slabih svetlobnih razmerah. Sistem je v temi ali senci manj zanesljiv, saj model včasih spregleda tarčo. Ker se kosilnica uporablja podnevi, to ni kritična omejitev.

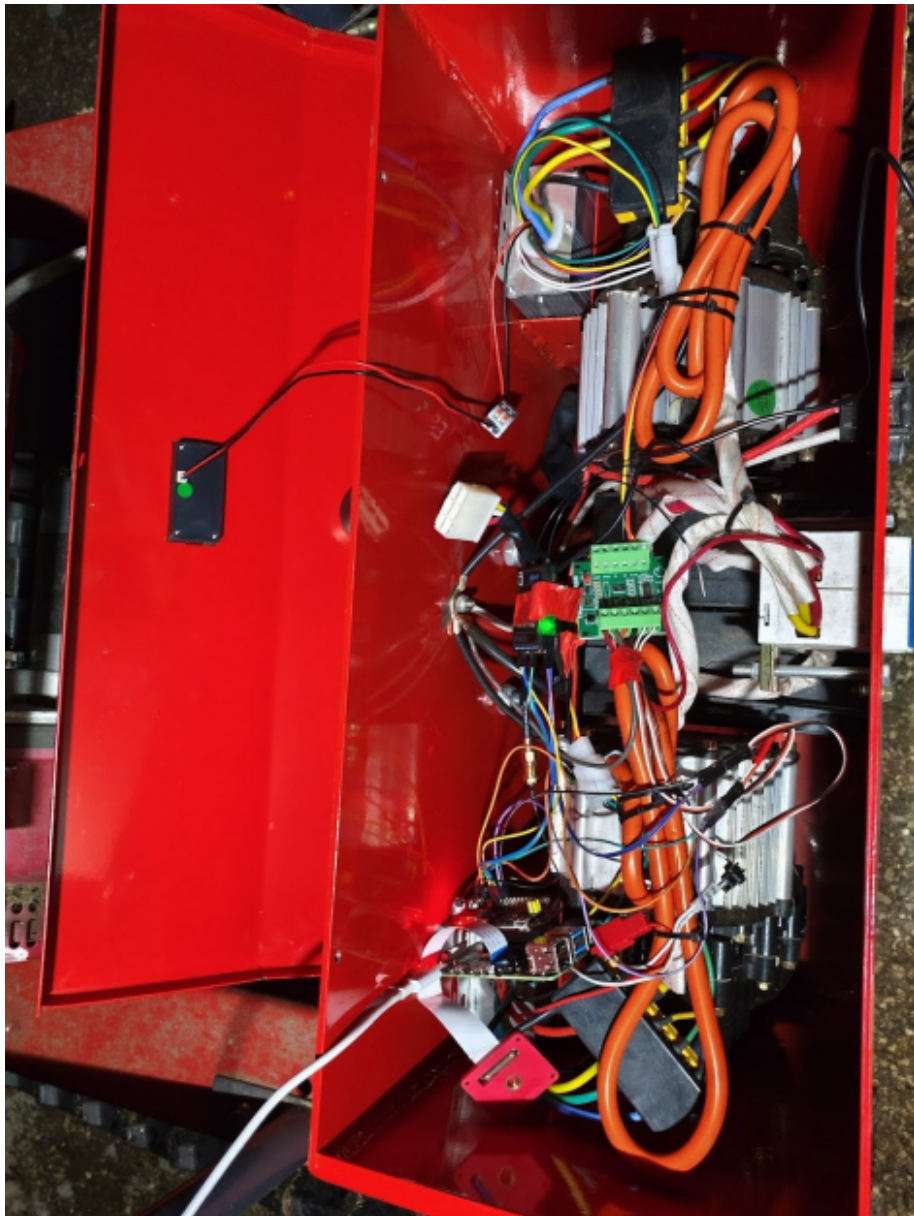
Natančnost GPS. Brez DGPS-popravkov je položaj odstopal tudi za več kot ± 5 m. Z uporabo NTRIP-popravkov se je natančnost izboljšala na približno $\pm 1,5$ m, vendar ni dosegla ciljne natančnosti pod 1 m, predvsem zaradi oddaljenosti referenčne postaje.

Omejitve zaznave na razdalji. Zanesljivost zaznavanja se pri razdaljah nad 4 m zmanjša, zlasti če je objekt delno zakrit. To je posledica uporabe zmanjšane ločljivosti slike za hitrejšo obdelavo.

Nastavitve PWM-signalov. Sprva je prihajalo do nestabilnega delovanja zaradi neustrezno nastavljenih mejnih vrednosti. Po prilagoditvi pragov je sistem deloval stabilno.



Slika 9: Testiranje sistema na terenu: mulčar od daleč s prenosnim računalnikom med preizkusi.



Slika 10: Notranjost mulčarja z dodanim računalnikom Raspberry Pi in nameščeno elektro-
niko.



Slika 11: Daljinec, nameščen na mulčarju med terenskim testiranjem.

4 Zaključek

V seminarski nalogi sem predstavil nadgradnjo daljinsko vodene kosilnice z računalniškim vidom, spletnim vmesnikom in natančnejšim določanjem položaja. Projekt sem zasnoval od izbire strojne opreme in programske arhitekture do implementacije glavnih funkcionalnosti: zaznavanja ljudi in živali, samodejne zaustavitve, prenosa video slike ter prikaza lokacije na interaktivnem zemljevidu.

Hipotezo, da je z računalniškim vidom mogoče zanesljivo zaznati ljudi in živali v okolici kosilnice ter preprečiti morebitne poškodbe, potrjujem. Sistem je v testnih razmerah dosegel odzivni čas pod 100 ms, kar je bistveno boljše od zastavljene meje 700 ms. Kljub temu ostajajo nekatere omejitve: zaznavanje je manj zanesljivo v slabih svetlobnih razmerah in pri večjih razdaljah.

Pri delu sem ugotovil, da so si posamezni deli sistema na prvi pogled videti razmeroma preprosti, v praksi pa se pri vsakem pojavijo nove težave. Posebej pomembno je bilo uskladiti hitrost zaznavanja, zakasnitev video prenosa in zanesljivost varnostne logike. Pokazalo se je, da pri takšnem sistemu ni najpomembnejša sama teoretična natančnost, temveč predvsem dovolj hiter odziv v resničnem okolju.

Skozi razvoj projekta sem pridobil praktične izkušnje z računalniškim vidom, večnitnim programiranjem, GNSS-lociranjem, spletnimi vmesniki in povezovanjem elektronskih modulov v enoten sistem. Za morebitno nadgradnjo bi bil smiselni novjš modul z AI kamero in

dodajanje luči za boljšo osvetlitev.

Viri in literatura

- European Space Agency. (2024). *What is GNSS?* Pridobljeno 6. aprila 2026, https://www.esa.int/Applications/Navigation/The_benefits_of_Galileo/What_is_GNSS
- Federal Agency for Cartography and Geodesy (BKG). (2024). *NTRIP – Networked Transport of RTCM via Internet Protocol*. Pridobljeno 6. aprila 2026, <https://igs.org/ntrip/>
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. Pridobljeno 6. aprila 2026, <https://arxiv.org/abs/1704.04861>
- LC29H (EA&DA) GNSS Module Series – Hardware Design Guide. (2023). Quectel Wireless Solutions. Pridobljeno 6. aprila 2026, <https://www.quectel.com/product/gnss-lc29h-series>
- OpenCV Team. (2024). *Deep Neural Networks (dnn module) – OpenCV Documentation*. Pridobljeno 6. aprila 2026, https://docs.opencv.org/4.x/d2/d58/tutorial_table_of_content_dnn.html
- Raspberry Pi Ltd. (2023). *The Picamera2 Library*. Pridobljeno 6. aprila 2026, <https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf>
- SNIP / RTK2go Community. (2024). *RTK2go – Free Public NTRIP Caster*. Pridobljeno 6. aprila 2026, <https://rtk2go.com>

1 Priloge

A.1 Video demonstracija sistema

Video demonstracija delovanja sistema z računalniškim vidom in GPS-lociranjem je dostopna na naslovu: https://drive.google.com/file/d/1oJzYue71dO5h15cRMGLaXBiq_ljRqebY/view?usp=sharing

A.2 Izvorna koda projekta

Celotna izvorna koda projekta je javno dostopna na repozitoriju GitHub: <https://github.com/lin-cadez/GardenMowerAutopilot/tree/main/code>

Izjava o avtorstvu

Izjavljam, da je seminarska naloga *Krmiljenje, regulacije, nadzor procesov* v celoti moje avtorsko delo, ki sem ga izdelal samostojno s pomočjo navedene literature in pod vodstvom mentorja.

13. april 2026

Lin Čadež